# JakDev — Technology Stack

## 1. Overview

JakDev is built as a modern, lightweight web application focused on providing a fast and simple free source-code library.

The technology stack is intentionally kept practical and maintainable.

Primary priorities:

* Fast development
* Simple architecture
* Good developer experience
* Responsive UI
* Interactive visual experience
* Secure admin management
* Easy deployment
* Low operational complexity
* Easy future maintenance

---

# 2. Core Stack

| Layer | Technology | Purpose |
| --- | --- | --- |
| Framework | Next.js | Application framework |
| Language | TypeScript | Type-safe development |
| Styling | Tailwind CSS | Utility-first styling |
| UI Components | shadcn/ui | Base reusable UI components |
| Icons | Lucide React | Interface icons |
| Animation | Motion | UI animation and interaction |
| Database | Supabase PostgreSQL | Application database |
| Authentication | Supabase Auth | Admin authentication |
| Storage | Supabase Storage | Preview assets and uploaded files |
| Deployment | Vercel | Production deployment |

Next.js is used as the primary application framework and is deployed through Vercel. Vercel provides native Next.js deployment support.

---

# 3. Frontend

## 3.1 Next.js

Use Next.js as the main application framework.

Responsibilities:

* Page routing
* Server and client rendering
* API/server functionality where required
* Metadata and SEO
* Application structure
* Admin dashboard
* Public library

Use the App Router architecture.

---

## 3.2 TypeScript

TypeScript is required throughout the project.

Use TypeScript for:

* Components
* Server functions
* Database types
* Forms
* API responses
* Resource models
* Utility functions

Avoid unnecessary use of `any`.

Create reusable types for:

* Resource
* Category
* Technology
* Responsive configuration
* Admin user
* Resource status

---

# 4. Styling

## 4.1 Tailwind CSS

Tailwind CSS is the primary styling system.

Use Tailwind for:

* Layout
* Spacing
* Typography
* Responsive behavior
* Colors
* Borders
* Shadows
* States
* Component styling

Avoid creating large amounts of custom CSS unless Tailwind cannot reasonably handle the requirement.

---

# 5. UI System

## 5.1 shadcn/ui

Use shadcn/ui as the base UI component system.

Recommended usage:

* Button
* Input
* Select
* Dialog
* Dropdown
* Tabs
* Tooltip

* Sheet
* Alert Dialog
* Form elements
* Tables
* Pagination

shadcn/ui components should be customized to match the JakDev visual identity.

Do not allow the default shadcn appearance to define the entire product.

---

## 5.2 Lucide React

Use Lucide React for interface icons.

Use icons for:

* Navigation
* Search
* Filters
* Copy
* Edit
* Delete
* Settings
* Categories
* Admin actions
* Responsive preview controls

Avoid mixing many unrelated icon libraries.

---

# 6. Interactive UI Libraries

JakDev should use external component libraries selectively to improve the visual quality and interaction of the interface.

These libraries are supporting resources, not the foundation of the application architecture.

---

## 6.1 React Bits

React Bits is an open-source collection of animated, interactive, customizable React components. It supports JavaScript/TypeScript and CSS/Tailwind implementations.

Use React Bits selectively for:

* Animated text
* Hero effects
* Interactive cards
* Scroll animations
* Background effects
* Particles
* Spotlight effects
* 3D-style visual effects
* Decorative interactive elements

Potential use cases in JakDev:

* Landing page hero

* Resource showcase
* Animated headings
* Interactive backgrounds
* Featured resource sections

Do not use React Bits components everywhere.

The interface must remain coherent.

Official resource:

[React Bits](https://reactbits.dev/?utm_source=chatgpt.com)

---

## 6.2 ScrollX UI

ScrollX UI is an open-source React UI library focused on customizable animated and interactive components. Its current stack is aligned with Next.js, TypeScript, Tailwind CSS, Lucide React, and Motion.

Use ScrollX UI selectively for:

* Interactive navigation
* Animated buttons
* Interactive cards
* Side sheets
* Motion-based UI
* Background effects
* Scroll interactions
* Decorative interactive elements

Examples include interactive components such as Motion Navbar, Stagger Button, Side Sheet, and Dot Wave.

Do not use ScrollX UI as a replacement for the entire JakDev design system.

Official resource:

[ScrollX UI](https://scrollxui.dev/?utm_source=chatgpt.com)

---

# 7. Animation

## 7.1 Motion

Use Motion for React animations and interaction.

Use it for:

* Page transitions
* Hover interactions
* Scroll animations
* Modal transitions
* Navigation animations
* Resource card interactions
* Micro-interactions

Animation principles:

* Keep animations subtle.
* Prioritize usability.
* Avoid excessive movement.

* Respect reduced-motion preferences.
* Do not animate every element.

Motion is also part of the technology foundation used by current ScrollX UI components.

---

# 8. Database

## 8.1 Supabase

Supabase is the primary backend platform.

Use:

* Supabase PostgreSQL
* Supabase Auth
* Supabase Storage

Supabase provides official Next.js integration and supports SSR through `@supabase/ssr`.

---

## 8.2 PostgreSQL

PostgreSQL is the primary relational database.

Core entities:

```text
categories
resources
technologies
resource_tags
admin_users
```

Keep the database schema simple.

Do not create unnecessary tables for features that are not part of the MVP.

---

# 9. Authentication

## 9.1 Supabase Auth

Supabase Auth is used only for administrative access.

Normal visitors do not need an account.

Admin authentication protects:

* Admin Dashboard
* Category management
* Resource management
* Resource publishing
* Resource deletion

Use `@supabase/ssr` for proper Supabase authentication handling in the Next.js server/client environment.

---

# 10. Storage

## 10.1 Supabase Storage

Use Supabase Storage for uploaded assets such as:

* Resource preview images
* Resource thumbnails
* Optional visual assets

Do not use Storage for the primary database records.

Resource metadata belongs in PostgreSQL.

---

# 11. Resource Source Code

Source code is stored as text data associated with a resource.

The MVP should keep source-code management simple.

A resource can contain:

```text
title
description
category
technology
tags
source_code
preview
responsive_support
status
```

Do not implement a complex repository/file-management system.

JakDev is not intended to become GitHub or a package registry.

---

# 12. Code Display

## 12.1 Syntax Highlighting

Use **Shiki** for source-code syntax highlighting.

Supported languages should include common web-development languages such as:

```text
HTML
CSS
JavaScript
TypeScript
JSX
TSX
JSON
Markdown
```

The implementation should remain extensible for additional languages.

---

# 13. Code Editor

A code editor may be used in the Admin Dashboard for editing source code.

Preferred options:

### Option A

**CodeMirror**

### Option B

**Monaco Editor**

Choose the lighter and more appropriate option for the actual MVP implementation.

The editor must support:

* Syntax highlighting
* Code editing
* Copying
* Basic formatting
* Language selection where appropriate

Do not introduce a full IDE experience.

---

# 14. Preview System

Resource previews should prioritize simplicity.

The system should support previewing HTML/CSS/JavaScript-based resources and suitable React-based resources where technically practical.

Use an isolated `iframe` approach where possible for client-side previews.

Important:

* Do not execute untrusted source code directly in the main application DOM.
* Keep preview execution isolated.
* Prevent preview code from accessing the main application environment.
* Sanitize or restrict dangerous operations where applicable.

The preview system must not compromise the security of the JakDev application.

---

# 15. Responsive Preview

Responsive preview is configured per resource.

Database representation should allow:

```text
desktop: true
tablet: true
mobile: false
```

Only enabled viewport options should appear on the public resource page.

Example:

```text
Desktop | Tablet
```

The system must not automatically claim that a resource supports responsive layouts.

The administrator controls the supported viewport options.

---

# 16. Search and Filtering

Search should initially be implemented using database queries and simple frontend filtering where appropriate.

Search fields:

* Title
* Description
* Category
* Technology
* Tags

Filtering:

* Category
* Technology

Sorting may include:

* Newest
* Oldest
* Alphabetical

Do not introduce a dedicated search engine during the MVP.

---

# 17. Forms and Validation

Recommended packages:

```text
react-hook-form
zod
```

Use:

### React Hook Form

For:

* Resource creation
* Resource editing
* Category creation
* Category editing
* Admin forms

### Zod

For:

* Form validation
* Server-side validation
* Resource schema validation
* API/server action input validation

Never rely only on client-side validation.

---

# 18. Notifications

Use:

**Sonner**

for lightweight feedback.

Examples:

```text
Resource created
Resource updated
Resource deleted
Code copied
Category created
Category deleted
```

Notifications should remain concise.

---

# 19. Utility Packages

Recommended utilities:

```text
clsx
tailwind-merge
```

Use these for clean conditional Tailwind class handling.

Avoid adding utility libraries when a small native TypeScript function is sufficient.

---

# 20. Data Fetching

Use Next.js server-side capabilities and Supabase directly where practical.

Do not introduce a large client-side state-management library for the MVP unless a real requirement appears.

Avoid unnecessary use of:

* Redux
* Zustand
* MobX

* React Query

Start with the simplest architecture that correctly handles the application's data flow.

---

# 21. Deployment

## 21.1 Vercel

Deploy JakDev to Vercel.

Vercel provides direct support for deploying Next.js applications.

Deployment responsibilities:

* Production hosting
* Preview deployments
* Environment variables
* Build process
* Application delivery

---

# 22. Environment Variables

Environment variables must be used for sensitive configuration.

Example:

```env
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=
```

Never expose:

```text
SUPABASE_SERVICE_ROLE_KEY
```

to the client.

Never hardcode secrets in source code.

---

# 23. Package Management

Use **npm** as the default package manager unless the project explicitly switches to another package manager.

Keep dependencies minimal.

Before installing a package, verify that it provides meaningful value.

Avoid dependency duplication.

---

# 24. Dependency Philosophy

JakDev follows this rule:

> **Install only what we actually need.**

Priority:

```text
Native Next.js / React
        ↓
Tailwind / shadcn
        ↓
Small focused library
        ↓
Large library only when justified
```

Do not install a package simply because it is popular.

---

# 25. External Component Philosophy

React Bits and ScrollX UI should be treated as component resources.

They may be:

* Used directly when appropriate.
* Adapted to JakDev's design.
* Used as inspiration for interactions.
* Reimplemented when a lighter custom solution is better.

Do not create unnecessary dependency coupling.

The core JakDev interface must remain maintainable even if a third-party component library changes.

---

# 26. Accessibility

The stack must support accessible interfaces.

Implement:

* Semantic HTML
* Keyboard navigation
* Focus states
* Accessible labels
* Proper button elements
* ARIA attributes when necessary
* Reduced-motion support
* Sufficient contrast

Third-party components must still be reviewed for accessibility before use.

---

# 27. Performance

Performance is a priority.

Use:

* Server rendering where appropriate

* Lazy loading
* Image optimization
* Code splitting
* Minimal client components
* Efficient database queries
* Optimized preview assets

Avoid unnecessarily shipping large JavaScript bundles to the client.

Interactive components should use client-side rendering only when required.

---

# 28. Security

Security priorities:

### Admin

Protect all admin routes.

### Database

Use Supabase Row Level Security where appropriate.

### Source Code

Treat submitted source code as untrusted content.

### Preview

Isolate executable preview content.

### Secrets

Never expose service-role credentials.

### Input

Validate all admin input on the server.

---

# 29. Development Structure

Recommended project structure:

```text
src/
├── app/
│   ├── (public)/
│   ├── admin/
│   └── api/
│
├── components/
│   ├── ui/
│   ├── layout/
│   ├── library/
│   ├── resource/
│   └── admin/
│
├── lib/
│   ├── supabase/
│   ├── validations/
```

```
│   ├── utils/
│   └── preview/
│
├── types/
│
└── styles/
```

Keep the structure understandable.

Do not create folders solely for architectural decoration.

---

# 30. Recommended Package Set

The initial package ecosystem should remain close to:

```text
next
react
react-dom
typescript

tailwindcss
shadcn/ui
lucide-react

motion

@supabase/supabase-js
@supabase/ssr

react-hook-form
zod

sonner
clsx
tailwind-merge

shiki

CodeMirror or Monaco Editor
```

Additional packages should only be added when an actual product requirement requires them.

---

# 31. Component Resources

Use:

```text
React Bits
ScrollX UI
shadcn/ui
```

as complementary component resources.

React Bits is especially useful for animated and visually distinctive React components, while ScrollX UI provides customizable interactive components and

motion-based UI patterns.

The final JakDev UI should combine these resources with custom components so the product maintains a consistent identity.

---

# 32. Design and Technology Principle

Technology must support the product, not become the product.

Avoid unnecessary complexity.

JakDev should remain:

```text
Simple
Fast
Maintainable
Interactive
Responsive
Secure
Developer-focused
```

The implementation should always prioritize the core experience:

```text
Discover
   ↓
Preview
   ↓
Copy
   ↓
Build
```

---

# 33. Final Stack Summary

```text
FRONTEND
Next.js
React
TypeScript
Tailwind CSS

UI
shadcn/ui
Lucide React

INTERACTION
Motion
React Bits
ScrollX UI

FORMS
React Hook Form
Zod

CODE
Shiki
CodeMirror / Monaco Editor
```

```
BACKEND
Supabase

DATABASE
PostgreSQL

AUTH
Supabase Auth

STORAGE
Supabase Storage

DEPLOYMENT
Vercel

UTILITIES
clsx
tailwind-merge
Sonner
```

## Final Rule

Do not over-engineer JakDev.

The technology stack exists to support one clear product goal:

> **A fast, simple, free source-code library where developers can browse, preview, copy, and use code immediately.**